

Weather Prediction

Train a DL model to predict temperature for the next day, based on three day window. Model gets data from last three days - outputs the predicted temperature for the next day.

Youngjun Son (48472-EX)
Emma Hope (48453-EX)

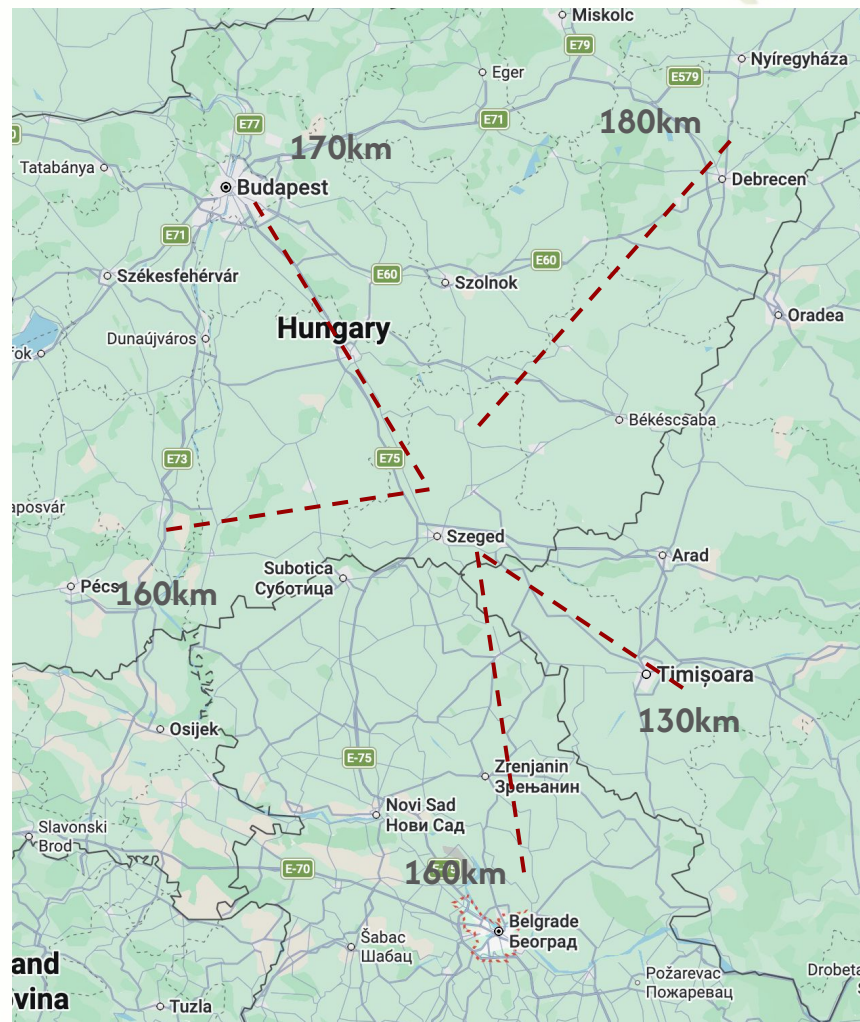


Step 0: Finding our dataset

Instead of relying only on the given dataset, we use the **Open-Meteo API** to extract 10 years of hourly weather data from 6 locations

Open-Meteo is an open-source weather API

-





Step 1: Cleaning and checking the data

Using

`.shape`

Returns a tuple representing the dimensionality of a DataFrame or Series, used to quickly understand the size of a dataset by providing the total count of rows and columns.

We checked for

- Data Completeness
- Feature Consistency
- Date Alignment

**+Check for missing columns
using `.isnull()`**



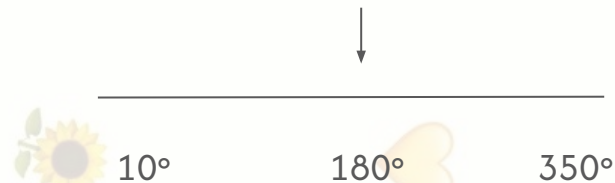
**Problem 1: Wind direction follows a 360°
compass and is not linear**

**Current
situation:**

Time	1pm	2pm	3pm
Wind direction	350°	?	10°



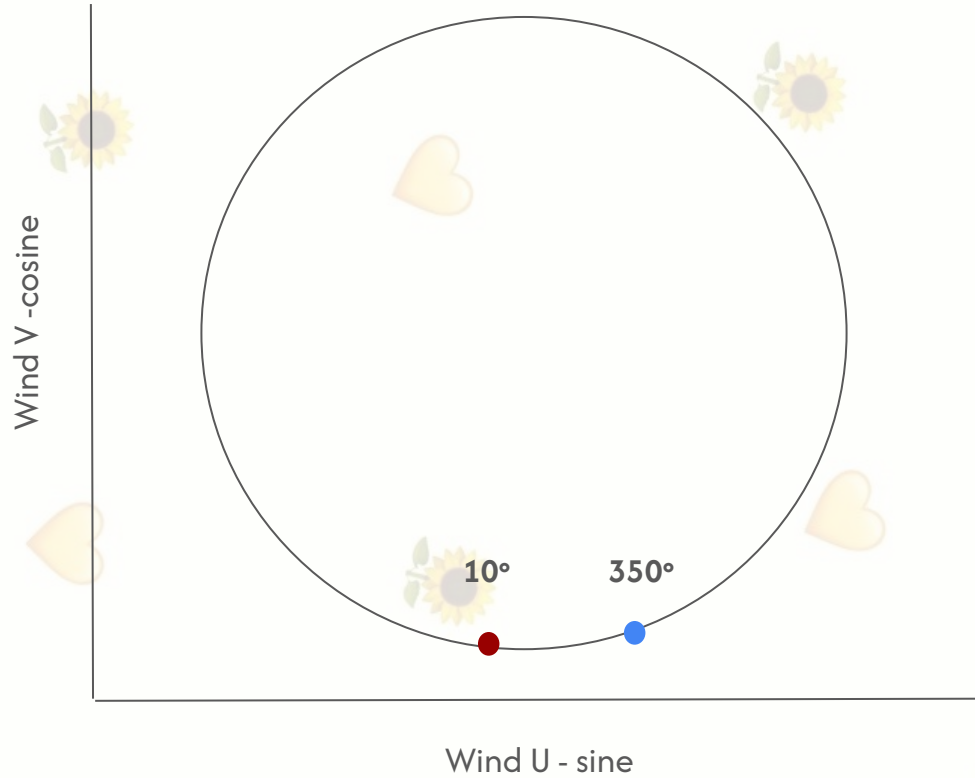
$$(350+10)/2 = 180^\circ$$



Solution: Convert `wind\_direction` and `wind\_speed` to `Wind U` using sine (East-West) and Wind V (North-South) using cosine

```
# 1. Convert wind to U/V vectors BEFORE interpolation (avoids 0/360 degree discontinuity)
theta = df['wind_direction_10m'] * np.pi / 180
df[f'{city}_wind_u'] = -df['wind_speed_10m'] * np.sin(theta)
df[f'{city}_wind_v'] = -df['wind_speed_10m'] * np.cos(theta)
```


Looks something like this:



Step 2: Splitting the data up + normalisation

74:14:12	80:10:10
Train, Val, Test	Train, Val, Test
0.017864	0.017103

Train/validation/test → 74:14:12

```
# Chronological hard split (full years for val/test to avoid seasonal bias)
train_mask = df_wide.index < '2014-01-01'
val_mask   = (df_wide.index >= '2014-01-01') & (df_wide.index < '2015-07-01')
test_mask  = df_wide.index >= '2015-07-01'
```

- **Train** = Apr 2006 – Dec 2013 (~74%)
- **Val** = Jan 2014 – Jun 2015 (~14%)
- **Test** = Jul 2015 – Sep 2016 (~12%)

Having a lower mse is good, but we wanted to split the data according to seasonal characteristics

Final Split: Chronological 74:14:12

Train on past weather, validate and test on future weather



Why this split?

Enough training history
Validation and test still cover future periods

Why Chronological?

1

Avoid future leakage

No information from future periods enters training.

2

Respect sliding windows

Adjacent samples overlap, so random split can inflate performance.

3

Realistic forecasting

The test set represents unseen future weather.



Step 3: Data preparation for LSTM model

Our parameters:

SEQ_LEN = 72 : The model can retrieve data from the past 72 hours to make a prediction

HORIZON = 24 : The model is to predict the weather over the next 24 hours

X (The Input): The 72 hours of past weather data

Y (The Target): The actual average temperature of the following 24 hours.

X+Y = sample window



python

```
def __getitem__(self, idx):  
    x = self.data[idx : idx +  
self.seq_len]  
    # Grabs the 72-hour clue  
    y = self.data[idx +  
self.seq_len : idx + self.seq_len  
+ self.horizon, self.target_idx]  
    y_mean = y.mean()  
    # Evaluates the 24-hour answer
```

Sample window 1	Hour 1-96
Sample window 2	Hour 97-192
Sample window 3	Hour 193-288

!! only ~700 samples to train on



Problem 3: How do we get more sample windows?

Sliding window

Sample window 1	Hour 1-96
Sample window 2	Hour 2-97
Sample window 3	Hour 3-98



~67,872 samples

python



```
def __getitem__(self, idx):  
    x = self.data[idx : idx +  
72]  
    y = self.data[idx + 72 :  
idx + 96, self.target_idx]
```

When $\text{idx} = 0$: it takes hours 0 to 72, and predicts 72 to 96.

When $\text{idx} = 1$: It takes Hours 1 to 73, and predicts 73 to 97.

When $\text{idx} = 2$: It takes Hours 2 to 74, and predicts 74 to 98.



Step 4: setting a benchmark

The benchmark provides a way to measure how much the deep learning model has improved.

python

```
def evaluate_baseline(dataset):
    preds, actuals = [], []
    for i in range(len(dataset)):
        x, y = dataset[i]
        last_temp = x[-1,
        target_idx].item() # Grabs the
        72nd hour's temperature

    preds.append(inverse_temp(last_tem
    p)) # Uses it as the guess

    actuals.append(inverse_temp(y.item
    ())) # Grabs the actual answer
```

This loop goes through every single sample window in the dataset. It looks at the very last hour of the 72-hour clue ($x[-1]$), saves it as its prediction (preds), and compares it against what actually happened (actuals).

python

```
mae = np.mean(np.abs(preds -
actuals))
rmse = np.sqrt(np.mean((preds
- actuals) ** 2))
r2 = r2_score(actuals, preds)
return mae, rmse, r2
```

Then, it calculates Mean Absolute Error, RMSE and R^2 to evaluate the outcome



Step 5: Model selection & comparison

**We use RNN model because weather data is a time series & RNN is the best for sequential data. Today's temperature is influenced by yesterday's/last week's temperature and past patterns are key to predicting the future.

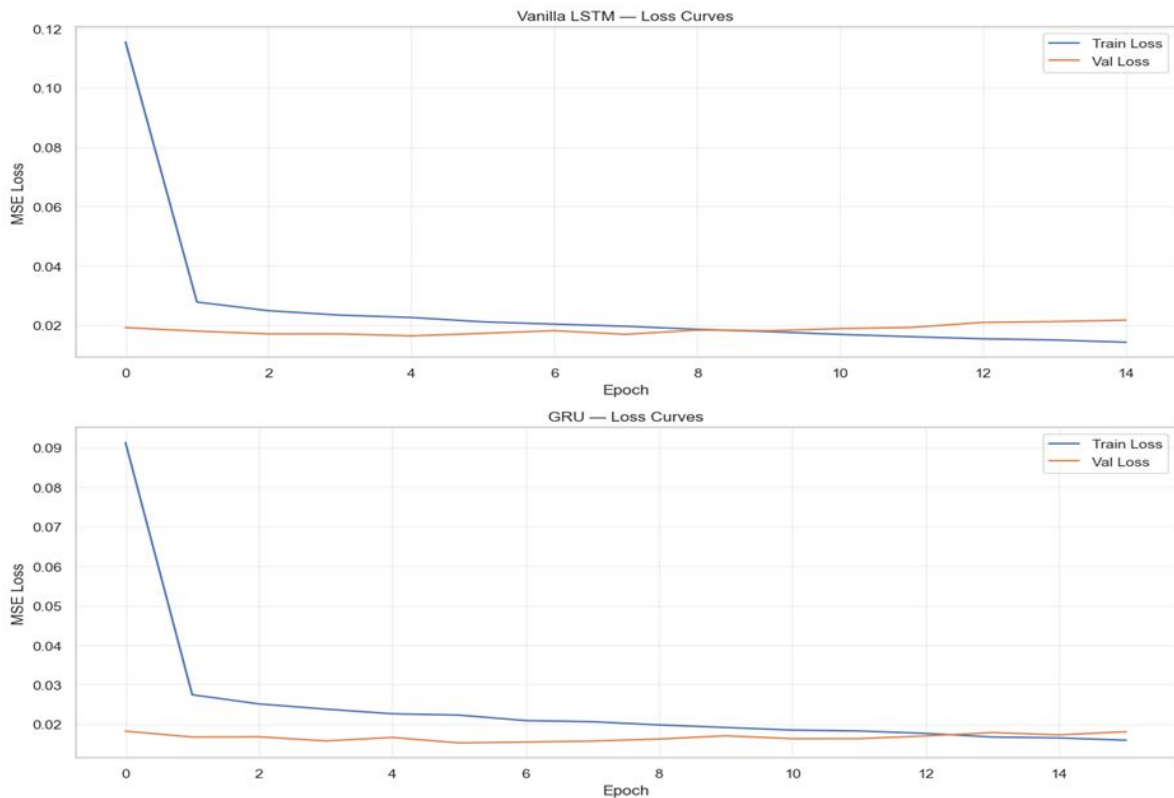
A Gated Recurrent Unit (**GRU**) is a streamlined version of the Long Short-Term Memory (**LSTM**) network

Vanilla LSTM	GRU
LSTM is designed to solve the vanishing gradient problem. It has a forget gate, allowing it to remember long-term patterns The most widely used model in the weather forecasting literature & is suitable as a baseline.	An efficient alternative to LSTM ~25% fewer parameters than LSTM which learns faster with the same data To test if simple > complex models

A Gated Recurrent Unit (**GRU**) is a streamlined version of the Long Short-Term Memory (**LSTM**) network

Vanilla LSTM	GRU
Uses 3 gates: Input, Forget and Output	Uses 2 gates: Update and Reset.
Has more parameters per cell	Has less parameters per cell (~25% Less)
Excellent at remembering long sequences	Good, but can struggle with extremely long-term dependencies compared to LSTM
Prone to overfitting if the data is sparse	Often performs better and is more robust on smaller datasets.
Better for complex climate modeling with many interacting variables.	Preferred for real-time temperature or rainfall alerts where speed is key.

Training Diagnostics



Key Takeaways

- Both models converge within ~6–10 epochs.
- Val Loss rises after optimum → Early Stopping effectively prevents overfitting.
- GRU achieves a slightly better and more stable minimum Val Loss.

Model Performance Comparison

Model	MAE (°C)	RMSE (°C)	R ² Score
Persistence (Baseline)	2.750	3.372	0.850
Vanilla LSTM	0.974	1.265	0.979
GRU (★ Best Model)	0.932	1.218	0.981



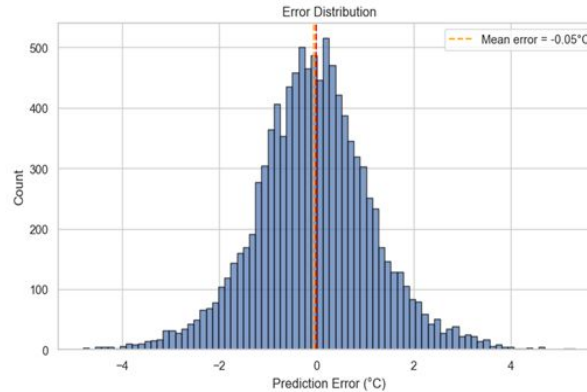
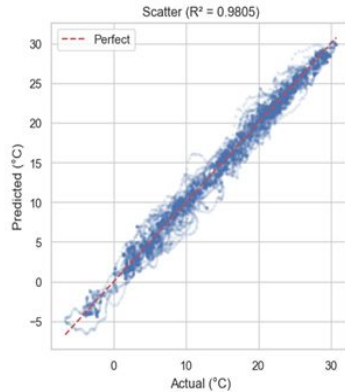
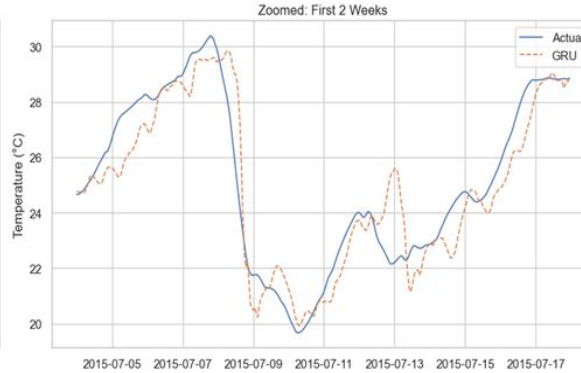
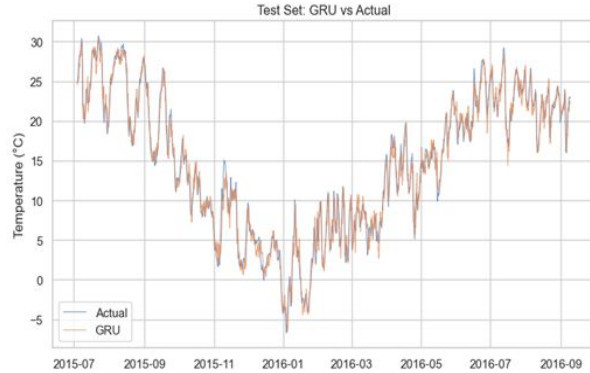
66% MAE Reduction
from 2.75°C to 0.93°C

GRU R² = 0.98

Explains 98% of the temperature variance

GRU Predictions

Best Model: GRU



Top Panels

Strong seasonal alignment.
Short lag on sudden spikes due to 72h historical reliance.

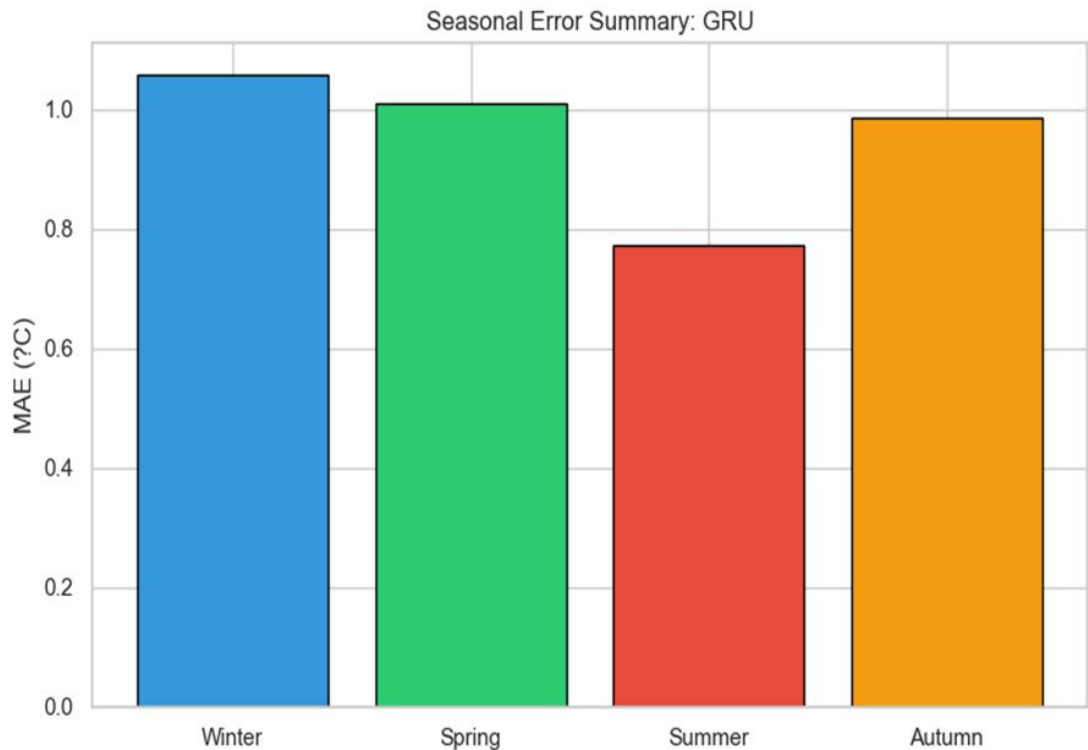
Bottom Left (Scatter)

High correlation ($R^2=0.98$) tight clustered around $y=x$.

Bottom Right (Err)

Normally distributed around 0.
Mean error = -0.05°C (no critical systemic bias).

Seasonal Error Analysis



Summer (0.77°C MAE)

Most accurate season due to stable, gradual temperature shifts.

Winter (1.05°C MAE)

Hardest season to predict. Model struggles slightly with sudden cold snaps.

Key Conclusions



Sliding Window

~67,872 training windows constructed chronologically.



GRU Outperforms

MAE 0.93°C vs LSTM's 0.97°C. Best overall metrics.



Robust Predictive Power

Explains 98% of the temperature variation.



Negligible Bias

Mean prediction error centered close to zero.

The **y axis** is the **North-South Motion** (wind v).
The **x axis** is the **East-West Motion** (wind u).

